

# Closing the Authority Gap: Combining Constraint Adherence (MTCP) with Causal Continuity (DecisionAssure)

Authors: Ahmad Abby (MTCP) & Akhilesh Warik  
(DecisionAssure) Date: May 29, 2026 Version: 1.0 License: No  
joint IP; each party retains ownership of its own  
framework. This note is a collaborative description of  
compatibility.

## 1. Two Independent Failure Domains

Modern AI governance architectures address two distinct failure domains, but often conflate them.

- **Constraint Adherence (MTCP domain):** Does the model's behaviour stay within the declared constraints during an interaction? MTCP evaluates whether the model holds its own constraints (e.g., output formatting, safety rules, scope boundaries) given the current context. It has no direct memory of the original authorisation event.
- **Causal Continuity (DecisionAssure domain):** Is the executing agent still mathematically identical to the observer that originally received authority? DecisionAssure tracks reference frame hashes (policy, delegation, external state) and observer identity hashes (agent, session, memory) across steps. If either

hash changes without re-authorisation, continuity is broken, and execution is denied.

Neither domain can replace the other. A model can perfectly adhere to its constraints while executing under a mutated, unauthorised observer. Conversely, continuity can remain intact while the model violates constraints. Both checks are required for end-to-end governance.

## 2. Schema Mapping (as agreed)

The full mapping is documented in the attached Schema Mapping.pdf. Key points:

- MTCP authority claim structure: {authority\_id, action, context, timestamp} – no link to original authorisation.
- DecisionAssure's genesis legitimacy\_state includes observer\_identity\_hash and reference\_frame\_hash, signed by the SCQOS gate.
- Each subsequent step links back to the previous valid state via previous\_observer\_identity\_hash and previous\_reference\_frame\_hash.
- The continuity\_valid boolean is true only if both hashes are unchanged from the last valid state.

## 3. Worked Example: Continuity Fracture Leading to MTCP Rejection

### Scenario

An AI agent is authorised at time T0 (genesis). It performs

two valid actions. Then, due to a policy mutation (or memory drift), the agent's `observer_identity_hash` changes without re-authorisation. The agent then makes a new claim to MTCP.

## DecisionAssure Trace (simplified)

### Step 1 – Genesis (authorisation)

```
json
{
  "step_name": "authorize",
  "observer_identity_hash": "dbc49ba9b0dd86a1",
  "reference_frame_hash": "cff9fa56f197e00b",
  "continuity_valid": true,
  "decision": "ADMIT"
}
```

### Step 2 – Action (continuity intact)

```
json
{
  "step_name": "tool_call",
  "observer_identity_hash": "dbc49ba9b0dd86a1", // same
  "reference_frame_hash": "cff9fa56f197e00b", // same
  "continuity_valid": true,
  "decision": "ADMIT"
}
```

### Step 3 – Mutation (e.g., policy change)

```
json
{
  "step_name": "policy_mutation",
  "observer_identity_hash": "95e1c0464a01bbd5", // changed!
```

```
"reference_frame_hash": "33a6d6e0f3c32522",    // changed!
"continuity_valid": false,
"decision": "DENY"
}
```

Step 4 – Agent attempts to make a new claim to MTCP (bypassing re-authorisation) Before MTCP evaluates the claim, the system injects the latest continuity\_valid flag from DecisionAssure: false.

## MTCP Evaluation (extended)

MTCP receives an extended claim structure:

```
json
{
  "authority_claim": {
    "authority_id": "agent_alice",
    "action": "release_funds",
    "context": { "session_id": "sess_123", "input_tokens": [...] }
  },
  "continuity_verification": {
    "continuity_valid": false,
    "last_valid_observer_hash": "dbc49ba9b0dd86a1",
    "current_observer_hash": "95e1c0464a01bbd5",
    "causal_continuity_persisted": false
  }
}
```

Because continuity\_valid == false, MTCP overrides its normal constraint evaluation and immediately returns:

```
json
{
```

```
"verdict": "DENY",  
  "reason": "Continuity fracture – observer no longer matches original  
authority."  
}
```

The model layer never sees the claim; the denial is deterministic and non-probabilistic.

## 4. Lightweight Integration Path

- No model retraining – MTCP remains unchanged; it only receives an additional input flag.
- Non-invasive – DecisionAssure runs as a separate trace engine; SCQOS (or any constitutional gate) provides the continuity\_valid flag.
- Integration steps:
  - 1 Before each MTCP evaluation, query DecisionAssure for the latest continuity\_valid for the current observer/session.
  - 2 If false, MTCP returns DENY without further evaluation.
  - 3 If true, proceed with normal MTCP constraint adherence check.
- Deterministic signal – continuity\_valid is a boolean, not a probability. It provides a hard floor for admissibility.

## 5. Non-Claims & Legal Safety

- This note describes compatibility, not a joint product. Each party retains full ownership of its own

framework.

- No commercial obligation arises from this note.
- The worked example is illustrative; real integration would require appropriate contracts.
- DecisionAssure traces are replayable and structured, but not yet cryptographically signed (planned extension). Tamper-evident signing would further strengthen the proof.

## 6. Transparent Profit Path (for reference, not obligation)

- Free – This technical note and the schema mapping are collaborative, no charge.
- Paid pilot (\$1,500, pay only if satisfied) – If the authors or their organisations wish to integrate DecisionAssure with MTCP in a real workflow, a pilot can deliver a working integration adapter + replayable trace.
- Referral – 15% of first invoice if a client is introduced and a deal closes.

These options are offered transparently, with no pressure. The note itself is valuable regardless.

End of joint technical note.